

IMPROVING STUDENT PERCEPTION AND CONFIDENCE USING PROGRAMMING IN MANUFACTURING ENGINEERING

Meet Upadhyay¹ and Casey Keulen¹

¹Department of Materials Engineering, University of British Columbia
Corresponding Author E-mail Address: meet.upadhyay@ubc.ca

Abstract – *Programming is increasingly integral to engineering education, yet students often struggle with its cognitive demands and unfamiliar logical frameworks. Python, with its intuitive syntax and practical utility, has shown promise in bridging this gap, particularly when applied to real engineering contexts where programming tasks are grounded in familiar disciplinary problems. This paper presents the design and implementation of Python-based Jupyter Notebooks integrated into a second-year manufacturing processes laboratory course, where students have prior but limited exposure to Python from earlier coursework. The Notebooks enable students to perform engineering calculations, visualize experimental results, and work through data processing tasks within the context of existing laboratory activities. They also provide instructors with a means to scaffold learning and guide laboratory activities. Instructional resources were created to take advantage of these opportunities. They were organized and hosted as an open educational resource using the Jupyter Book framework and GitHub Pages to support independent student access throughout the term. Formative survey feedback indicated that students found the notebooks accessible and practical, and most expressed intent to apply Python in future engineering work. Students also identified areas for improvement, particularly around opportunities to write code themselves and the clarity of inline documentation. These findings highlight a core design tension between scaffolding accessibility and skill development and inform recommendations for educators seeking to embed computational tools within discipline-specific laboratory courses.*

Paper Type: Educational Practice

Keywords: manufacturing laboratory, programming, computational thinking, jupyter notebooks, surveys, open educational resources

1. INTRODUCTION

Programming and data analysis skills are increasingly essential for engineering practice, particularly in fields such as manufacturing where engineers must monitor and

interpret data from machine sensors, perform simulations, and automate calculations [1]. As a result, many engineering programs integrate high-level programming languages such as Python into core courses [2]. While this integration has increased student exposure to programming, it has not consistently led to widespread adoption of programming as a routine problem-solving tool. Students often encounter programming in isolated contexts and may continue to rely on more manual calculations or spreadsheet-based tools even when computational tools would be beneficial.

Sources state that students frequently experience distress and frustration when learning to program, primarily due to the cognitive demands and the necessity to develop new logical thinking patterns and competencies [3, 4]. A significant hurdle faced is the inherent difficulty and perceived tediousness of programming, which demands continuous cognitive effort and the ability to persist through frequent errors and debugging processes [5]. This learning curve can be particularly steep in traditional programming courses, which often emphasize laboratory-based, self-driven problem-solving, sometimes leading to high failure rates [6]. Efforts to improve programming education have included various pedagogical techniques, such as pair programming, peer instruction, and media computation. Despite these strategies, students still struggle with developing programming competencies. Additionally, the choice of programming language plays a crucial role in shaping students' initial experiences with coding. Studies suggest that high-level languages like Python, which offer intuitive syntax and robust functionality, may be particularly beneficial for engineering students [3].

The use of Python in educational settings has shown promise due to its balance of practical utility and algorithmic clarity [7]. Python's syntax allows students to focus more on problem-solving and theoretical concepts rather than getting bogged down by intricate language details, as often happens with languages like C or C++. Furthermore, Python's growing popularity in academia and industry underscores its relevance and potential as a teaching tool [7]. Cardoso, Leitão, and Teixeira [8] demonstrated that Jupyter Notebooks can serve as effective

teaching and learning tools in informatics engineering courses, finding that the combination of text, code, and visual outputs in a single browser-accessible document improved both teacher productivity and student flexibility in engaging with course materials. Building on this, Johnson [9] provided a systematic examination of the strengths and limitations of Jupyter Notebooks in the classroom, drawing on five years of teaching experience across multiple courses. While identifying clear advantages in terms of the notebook's readability and suitability for combining explanation with executable code, Johnson cautioned that notebooks can behave unpredictably, may encourage poor coding practices, and can create a passive learning dynamic if students rely too heavily on prewritten content without developing independent coding skills. These concerns are consistent with student feedback observed in the present study. Barba et al. [10] offer a broader pedagogical framework for teaching with Jupyter Notebooks across STEM disciplines, emphasizing the value of narrative-driven, executable documents for supporting active learning and reproducible workflows. More recently, Seddighi [11] demonstrated the effectiveness of combining Jupyter Notebooks with industry-standard software packages in a graduate-level engineering course, reporting improved student engagement through a blended learning structure in which notebooks supported both lecture content and practical assignments. Taken together, these studies support the approach taken in the present work, in which notebooks were designed to bridge the gap between prior programming exposure and applied use within a discipline-specific laboratory context.

Within the Manufacturing Engineering curriculum at the authors' institution, Python has been introduced across multiple courses to support numerical problem solving and laboratory data analysis. Despite this exposure, instructors have observed variability in student confidence and willingness to use programming independently for coursework, suggesting that exposure alone is not sufficient to support meaningful adoption. Students who encounter programming in contexts where hand calculations remain the assessed norm have limited incentive to develop Python fluency and may arrive in subsequent courses with familiarity but not confidence.

This work addresses that gap through the design and implementation of Jupyter Notebook resources integrated directly into the laboratory activities of a second-year manufacturing processes course. Rather than introducing programming as a standalone skill, the notebooks embed Python within the analytical workflow of existing laboratory tasks, providing structured examples and step-by-step workflows that connect code to disciplinary concepts. The paper describes the development process, implementation context, and student feedback, with the aim of offering practical insights for engineering educators

seeking to support programming adoption in laboratory and problem-solving contexts.

2. PRACTICE OBJECTIVES

2.1. Practice focus

The primary focus of this project is the development and implementation of Jupyter Notebook-based instructional resources to support the use of Python for data processing and visualization in a manufacturing engineering course. Rather than testing a formal hypothesis, the guiding design question for this work is:

Can structured Jupyter Notebook resources be designed and integrated into existing course activities to support students in using Python more confidently and consistently for engineering problem solving?

This question reflects the practical goal of supporting programming adoption within the course context by embedding computational tools into laboratory and problem-solving workflows.

2.2. Intervention

This project describes the development and implementation of Jupyter Notebook resources integrated into an existing second-year manufacturing processes laboratory course. Several approaches to supporting programming adoption were considered, including additional lectures on syntax, standalone tutorials, and optional coding assignments. These were set aside because they risked further isolating programming from disciplinary context, reinforcing the pattern of exposure without application that motivated this work. Instead, the instructional team developed notebook resources embedded directly within existing laboratory activities, providing scaffolded code templates, worked examples, and visualization tools aligned with the analytical tasks students were already required to complete in their lab reports.

The notebooks were introduced as optional but actively supported tools, with the aim of helping students move from prior programming exposure toward practical and confident, applied use within the course. Formative survey feedback was collected at two points during the term to inform ongoing refinement of the materials, particularly with respect to clarity of documentation, level of scaffolding, and alignment with course tasks.

2.3. Intended Objectives

The notebook development and integration were intended to support several interconnected instructional goals, each responding to observed gaps between student exposure to programming and their willingness to apply it independently in coursework.

- Encourage Python as a powerful option for laboratory data processing and visualization by

providing resources that make Python a practical and accessible alternative to Excel or manual calculation.

- Reduce the initial barrier to applying programming by offering structured, pre-populated code templates that students can engage with without needing to write code from scratch.
- Support student confidence in using Python for common analytical tasks, recognizing that confidence in applying a tool is often a prerequisite for adopting it independently.
- Improve students' perception of Python as a relevant and useful tool for engineering analysis, by grounding all examples and workflows in manufacturing engineering contexts.
- Provide a foundation for iterative improvement of the resources based on student feedback, with the aim of gradually increasing opportunities for independent coding as student familiarity grows.

These outcomes are framed as instructional aims of the practice rather than formal measurable treatment effects and are evaluated formatively through survey feedback collected during the course.

2.4. Rationale

The design of the Jupyter Notebook resources was informed by established educational theories and prior work in programming education.

Constructionist learning theory [12, 13] suggests that students develop understanding most effectively when actively engaged in creating and applying knowledge in authentic contexts. Integrating programming tasks into laboratory analysis provides an opportunity for students to apply computational tools directly to engineering problems, reinforcing both conceptual understanding and practical skill development.

Expectancy-Value Theory [14] provides a complementary perspective on student motivation. Students are more likely to engage with learning activities when they perceive them as both valuable and achievable. In this context, notebook resources were designed to clarify the relevance of Python to engineering tasks while providing structured support that increases students' expectations for success. By offering guided examples, visualization tools, and clear workflows, the notebooks aim to reduce uncertainty and support gradual development of independence.

Together, these perspectives informed the design of the notebook resources as scaffolded, context-integrated tools intended to support the transition from exposure to adoption of programming in engineering coursework.

3. METHODS

3.1. Context of MANU 230

MANU 230 is a second-year manufacturing processes laboratory course delivered across both the first and second term of second year. Engineering students have a common first year, which makes second year is the student's first year in the program. The course involves one hour of lecture and two hours of laboratory sessions per week. The cohort is approximately 50 students split across two lab sections of 25 students each. They typically work in groups of five during laboratory sessions and submit individual reports following each activity. The course covers a broad range of manufacturing processes focusing on metals, polymers, and composites. Examples of processes performed include casting, welding, deep drawing, spring winding, rolling and heat treating, injection moulding, thermoforming, 3D printing, vacuum infusion, prepreg processing, and resin transfer moulding (RTM). These activities provide students with unique hands-on exposure to processes many other programs do not offer.

While in the first term, most students also take MANU 261, a thermodynamics course that introduces Python through the CoolProp library [15] for fluid property calculations, among other libraries. However, because MANU 261 assessments require hand calculations, students have limited practical incentive to develop Python fluency in that context. As a result, students may have some familiarity with Python syntax from that course but are unlikely to have developed confidence or habits around using it independently for data analysis tasks.

Within MANU 230, students are not required to use Python and are free to complete data analysis and plotting using whichever tools they prefer, including calculators, MS Excel, online plotting tools, MATLAB, etc. This flexibility reflects the broader course priority of understanding manufacturing processes rather than focus on developing programming skills. Teaching assistants support students during laboratory sessions and evaluate submitted reports but are not expected to provide guidance on data analysis methods or Python specifically. In this context, the Jupyter Notebook resources were introduced as optional but structured supports, with the spring winding laboratory representing the one case where notebook use was more strongly encouraged due to the iterative nature of the spring parameter calculations involved.

3.2. Platform and delivery

The Python resources were implemented as a collection of Jupyter Notebook modules organized within a course website built using the Jupyter Book framework [16] and deployed via GitHub Pages. This approach provided a freely accessible, browser-based interface for students to view instructional content and download notebooks without requiring institutional login or specialized

infrastructure. These pages can be found at the following URL: <https://manu-ubc.github.io/manu230/intro.html>

Jupyter Book was selected because it supports the integration of executable notebook content with explanatory text, figures, and formatted documentation within a single navigable website. The framework allows notebooks to be displayed with pre-generated plots, tables, and worked examples while still allowing students to run and modify the code locally. Each page of the site includes options for downloading the notebook file for use in a local Python environment or launching it directly in Google Colab, providing flexibility for students with varying levels of programming experience and access to computing environments. Content is organized through a configuration file and table of contents, compiled into a static website using the Sphinx documentation engine and rendered as accessible, browser-readable HTML.

Deployment via GitHub Pages provides a cost-free, publicly accessible hosting solution. The site is automatically rebuilt and redeployed upon each content update using GitHub Actions, supporting a continuous integration workflow that facilitates collaborative and iterative development of instructional materials [17]. This infrastructure positions Jupyter Book as a practical framework for open educational resources in engineering education, with course materials remaining version-controlled and openly accessible throughout the term. The open structure of the site also supports reuse and adaptation of materials in future course offerings or related courses within the program.

each module grounds its examples in manufacturing engineering contexts drawn from the course itself. For instance, the basic operations module introduces variables and data types using injection moulding parameters such as mold temperature and cycle time, demonstrates conditional logic using vacuum infusion data, and illustrates plotting using a tensile test stress-strain dataset. This deliberate alignment between programming concepts and course content was intended to help students recognize the relevance of Python to their disciplinary work from the outset. The modules also include a brief section on the use of generative AI tools for coding assistance, encouraging students to engage critically with AI-generated code rather than relying on it passively.

Over the course of the term, students complete ten laboratory activities and submit individual reports for each. Jupyter Notebook resources were developed for four of these activities, representing the labs most amenable to programmatic data processing and visualization given the nature of the measurements and calculations involved. The lab-specific notebooks were developed for: vacuum infusion, prepreg processing, injection moulding with tensile testing, and spring winding. Each notebook mirrors the analytical workflow required in the corresponding laboratory report, guiding students through loading their experimental data, performing calculations, and generating plots of their results. Approximately 90 percent of the code in each notebook is pre-populated, with students required to enter their own measured variable values, load their own data files, and execute the cells to produce outputs. This

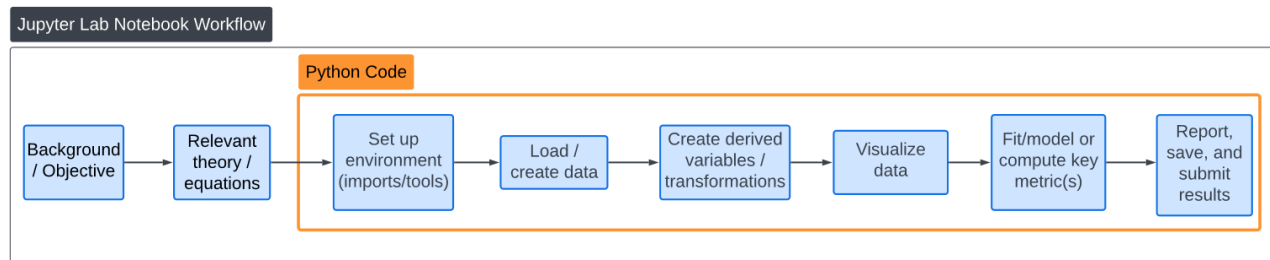


Figure 1: General flowchart of a Jupyter notebook. The theory and background sections are implemented using a combination of markdown and Latex and the Python Code section is a combination of markdown, code cells and output cells.

3.3. Implementation

The Jupyter Book site was organized into two components: a set of foundational Python instruction modules and a collection of lab-specific application notebooks. This structure was intended to allow students to build basic familiarity with the Python environment before engaging with process-specific analytical tasks.

The foundational modules were developed specifically for the course and cover Python environment setup, basic operations, and data processing using the Pandas library. Rather than presenting abstract programming exercises,

design reduces the initial barrier to entry by eliminating the need to write code from scratch while still requiring students to engage meaningfully with the structure and logic of the notebook. The spring winding notebook represents the most structured application of this approach, as it includes iterative calculations for spring parameters such as number of turns that are particularly well suited to programmatic computation and would be tedious to perform manually.

As illustrated in Figure 1, each lab notebook follows a consistent general structure progressing from process context and library imports through data input, calculation, and visualization to results interpretation. This consistent layout was intended to reduce cognitive overhead as students moved from one lab to the next, allowing them to focus on the engineering content rather than relearning the notebook environment for each experiment. Each notebook opens with a theoretical overview of the relevant manufacturing process, written in Markdown and structured using hierarchical headings that Jupyter Book automatically compiles into a navigable table of contents for the site. Process equations are rendered using LaTeX directly within the notebook, allowing students to see the governing relationships alongside the code that

implements them, reinforcing the connection between theory and computation. Experimental setup images are embedded within the notebooks to provide visual context for the data students are working with. Admonition blocks, a Markdown feature that renders highlighted callout boxes, are used throughout to draw student attention to key steps, common errors, or important parameter choices. Together these features allow each notebook to function as a self-contained instructional resource that integrates process theory, visual context, and executable analysis within a single document. Examples of these features are shown in Figure 2.

Theory

Before analyzing the data, a derivation of an equation to calculate the time required to infuse a laminate is needed. Please refer to the figure below for an explanation of the variables

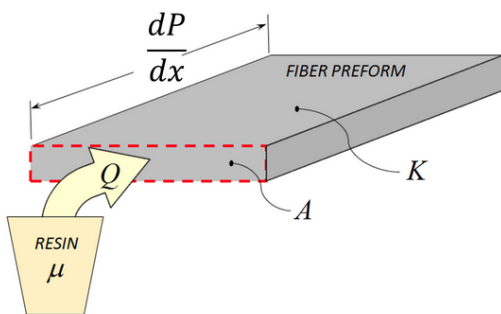


Fig. 1.1 Schematic of Darcy's Law

(a)

Equation (8) can be rearranged and integrated to solve for time:

$$\int_0^x x dx = \frac{K}{\mu \phi} (P_0 - P_x) \int_0^t dt$$

$$\frac{x^2}{2} = \frac{K}{\mu \phi} (P_0 - P_x) t$$

$$t = \frac{x^2 \mu \phi}{2K(P_0 - P_x)} \dots (9)$$

Equation (9) can be used to calculate the time required to fill a rectangular laminate and can be useful during design. However, in this lab, the objective is to determine the value of permeability, K . If one plots flow front position squared vs time, the slope of that line can be substituted into Equation (9) to get:

$$K = \frac{\mu \phi}{2(P_0 - P_x)} m \dots (10)$$

(b)

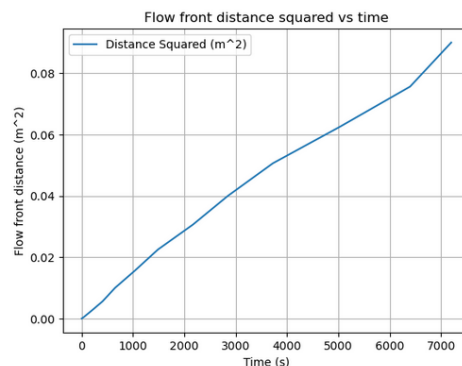
Think about it!

The strain at the break is different for the two specimens. Why is that? How does the gate design affect the strain at the break?

(c)

```
# Plot the DataFrame
df.plot(x="Time (s)", y=["Distance Squared (m^2)"], kind="line")

# Show the plot
plt.title("Flow front distance squared vs time")
plt.xlabel("Time (s)")
plt.ylabel("Flow front distance (m^2)")
plt.grid()
plt.show()
```



(c)

```
# Create a column of distance squared
df["Distance (m)"] = df["Distance (mm)"] / 1000
df["Distance Squared (m^2)"] = df["Distance (m)"] ** 2

# Display the data
df
```

	Time (s)	Distance (mm)	Distance (m)	Distance Squared (m^2)
0	0	0	0.000	0.000000
1	51	25	0.025	0.000625
2	188	50	0.050	0.002500
3	407	75	0.075	0.005625
4	649	100	0.100	0.010000

(d)

Figure 2: Representative panels from the vacuum infusion Jupyter Notebook illustrating the key features of the lab notebook design. (a) The theory section opens with an embedded schematic of the experimental setup. (b) LaTeX-rendered equations present the governing process physics alongside explanatory text. (c) A representative code cell with inline comments guides students through plotting their experimental data. (d) Pandas is used to load and display student measurement data in a structured table, with derived columns computed automatically. (e) An admonition block highlights a conceptual question for students to consider when interpreting their results. The notebook is available at <https://manu-ubc.github.io/manu230/intro.html>.

3.4. Design Features Supporting Student Use

The design of the lab notebooks was guided by a deliberate balance between accessibility and engagement. A central concern was that students accustomed to Excel or online plotting tools would revert to those familiar approaches if the Python-based alternative required significant coding effort to get started. To address this, notebooks were kept largely complete, with approximately 90 percent of the code pre-populated, so that students could produce meaningful outputs by entering their own data and running existing cells rather than writing code from scratch as shown to be effective in [9]. The goal was not to avoid programming entirely but to lower the threshold sufficiently that students would choose to engage with the notebooks rather than default to more familiar tools.

Code simplicity and readability were prioritized throughout. More advanced features such as interactive widgets or sliders were deliberately excluded, not because they lack educational value, but because they introduce complexity that can obscure the underlying logic and make it harder for students to follow what the code is actually doing. Instead, each notebook was written using straightforward operations with clear inline comments, so that a student reading through the code could trace the analytical steps without needing prior programming experience.

The growing availability of generative AI coding tools among students also informed the design philosophy of the notebooks. Engineers and engineering students increasingly encounter code they did not write themselves, whether generated by a colleague, adapted from an online resource, or produced by a large language model. Reading, interpreting, and modifying pre-written code is therefore a practically relevant skill. The pre-populated structure of the lab notebooks places students in precisely this position, requiring them to understand the logic of existing code, identify where their own data and variables need to be substituted, and troubleshoot errors in a context they did not fully author. This approach reflects a broader goal of familiarizing students with the practice of reading and interpreting existing code, a skill that is increasingly relevant in engineering contexts where Python scripts and analytical tools are shared and adapted across teams.

3.5. Evaluation

Two brief surveys were administered during the term using Qualtrics to gather formative feedback on student experiences with Python and the notebook resources. Likert-scale items were adapted from a similar instrument used in a comparable Python-based engineering laboratory context [18]. Ethics approval through the institutional Research Ethics Board delayed the first survey, which was administered approximately mid-term rather than at the start of the course as originally intended. The second survey was administered at the end of term. Thirteen

students (24 % of enrolled students) completed the first survey and eight (15 % of enrolled students) completed the second. Given the small cohort size (55) and the mid-term administration of the first survey, the two datasets are treated as independent snapshots of student experience rather than a formal pre-post comparison.

Both surveys included a combination of Likert-scale and open-response questions. The Likert items asked students to rate their comfort performing basic calculations in Python, their ability to create plots and analyze data, the usefulness of the examples and resources provided, the level of support received from instructors, and the likelihood that they would use Python in future engineering work. Open-response prompts invited students to describe challenges encountered when using Python, aspects of the notebooks they found helpful, and suggestions for improvement.

Survey responses were analyzed descriptively to identify trends in confidence and perceived usefulness of the resources, and to surface recurring themes in student feedback. Given the exploratory nature of this evaluation and the sample size, findings are interpreted qualitatively and used primarily to inform future iterations of the notebook design rather than to draw generalizable conclusions.

4. RESULTS AND DISCUSSION

Survey responses from both the mid-term and end-of-term surveys provided formative insight into how students engaged with the notebook resources and where the design could be improved. Results are interpreted descriptively given the small sample sizes of thirteen and eight respondents respectively.

End-of-term Likert responses indicated generally positive student perceptions of the notebook resources. Students reported strong comfort with performing basic calculations in Python (mean 4.38/5) and found the examples and tutorials provided in class useful (mean 4.38/5). Comfort with creating plots and analyzing data was moderately high (mean 3.77/5 for both), and most students indicated they intended to use Python for future engineering work (mean 4.00/5). Perceived support from instructors and TAs was also moderately positive (mean 3.92/5), though the higher standard deviation on this item (1.21) suggests variability in how supported individual students felt, which is consistent with the limited Python-specific support available from TAs in this course.

When asked where they had previously encountered Python, the majority of respondents indicated they had used it only within the Manufacturing Engineering program, with several noting exposure through mathematics courses such as MATH 255 and a physics lab course. This confirms that students arrive in MANU 230 with some prior exposure but through contexts that do not strongly reinforce independent applied use. Generative AI

tools were the most reported resource students turned to when writing code, used by 11 of 13 respondents, followed by classmates and friends and other online resources such as Stack Overflow. This pattern of help-seeking is worth noting as it reflects the realities of how students navigate programming challenges independently and suggests that the notebooks' readable, well-commented structure plays an important role in helping students interpret and adapt AI-generated suggestions.

Open-response feedback highlighted several consistent themes. Students valued the pre-populated structure of the notebooks, with several noting that having the code largely written allowed them to focus on understanding the analysis rather than troubleshooting syntax. One respondent noted that the prewritten code saved considerable time and produced reliable results, while another appreciated having all data and calculations in one place. At the same time, students acknowledged that this structure limited their development of independent coding skills, with one respondent explicitly noting that while the notebooks helped with completing assignments, they did not improve their Python abilities. This tension between accessibility and skill development was one of the central trade-offs of the design approach and is consistent with concerns raised in the literature regarding passive engagement with prewritten notebook content [9].

Students identified several areas for improvement. The most common requests were for clearer explanations of individual code blocks, more opportunities to write portions of code themselves, and additional guidance on how to modify provided templates for their own datasets. The mid-term survey similarly indicated interest in learning more about plotting, Pandas data frames, and general Python usage, with most respondents finding the notebooks useful or somewhat useful. One student suggested that interactive exercises like those used in another course in the program could be effective for building foundational skills, pointing to the potential value of complementing the notebook resources with more structured coding practice.

From the instructors' perspective, students were generally curious about Python and receptive to the notebook resources but tended to default to Excel or online plotting tools when those felt more immediately accessible, particularly for simpler calculations where the overhead of opening and running a notebook felt unnecessary. A dedicated in-class lecture and demonstration was conducted to walk students through data analysis workflows in Python, but this format proved difficult to execute effectively. Live coding in a classroom setting makes it easy for students to disengage, and students who fell behind at an early step, such as understanding how a variable is defined or how a data file is loaded, found it very difficult to recover as the demonstration progressed to more complex operations. This experience reinforced the value of self-paced asynchronous resources like the Jupyter

Book site, where students can pause, re-read, and re-run cells at their own pace, and suggested that in-class time is better used for briefly demonstrating the notebooks in context and answering specific questions rather than attempting to teach Python syntax directly.

5. RECOMMENDATIONS

5.1. Recommendations for Educators

The most effective notebooks were those designed around analytical tasks students were already required to complete in laboratory reports. When notebook workflows diverge from course deliverables, students tend to disengage, so alignment between the resource and the assessment is essential from the outset.

A graduated scaffolding approach is preferable to a uniform level of pre-population across all notebooks. Early notebooks can be largely complete to lower the initial barrier, while later notebooks should require students to fill in increasing portions of code themselves. This balances accessibility for less experienced students with meaningful skill development over the course of the term, and directly addresses the tension identified by students between the convenience of prewritten code and the limited opportunity to develop independent coding skills. Complementing notebook resources with structured coding exercises, for example short, guided tasks where students write or modify small portions of code themselves, would address the most consistent gap identified in student feedback. Several students pointed to interactive exercise platforms used elsewhere in the program as a model worth emulating.

Code simplicity and documentation quality should be treated as first-order design priorities. Students consistently requested clearer explanations of individual code blocks and better guidance on how to modify templates for their own datasets. Writing notebooks in plain, readable code with thorough inline comments requires deliberate effort but has a direct impact on whether students can engage with the material independently.

Given that generative AI tools were the most used resource when students encountered coding difficulties, notebooks should acknowledge and scaffold this practice explicitly. Including guidance on how to prompt AI tools effectively, and how to critically evaluate and adapt AI-generated code, prepares students for how programming is increasingly practiced in professional engineering contexts and reduces the risk of passive copy-paste behaviour.

Notebooks left as passive optional resources without in-class demonstration or active referencing by instructional staff will see limited uptake. Even where TAs are not expected to provide Python-specific support, briefly demonstrating notebooks during class and framing them as the expected approach for data analysis tasks significantly increases engagement.

Finally, hosting resources openly via GitHub Pages and maintaining them under version control reduces

administrative overhead across course offerings and makes materials straightforward to share and adapt. This is particularly valuable in programs where multiple courses could benefit from shared foundational Python resources, reducing duplication of effort across the curriculum.

5.2. Future directions

We intend to apply these recommendations directly to the next iteration of the MANU 230 notebook resources. Planned developments include introducing graduated scaffolding across the four lab notebooks so that later notebooks in the term require students to complete a greater proportion of the code themselves, improving inline documentation based on the specific gaps identified in student feedback, and exploring the addition of short guided coding tasks that complement the existing pre-populated structure. Survey administration will also be refined to allow for a more reliable pre-post comparison. Additionally, we plan to conduct interviews with students at the end of the course to provide more context to their survey responses. These iterative improvements will be evaluated through continued formative feedback collection, with the goal of progressively increasing student coding independence while maintaining the accessibility that made the current resources effective.

6. CONCLUSIONS

This work presented the design and implementation of Python-based Jupyter Notebook resources integrated into a second-year manufacturing engineering laboratory course. The central design question guiding this work asked how structured notebook resources could be designed and embedded into existing course activities to support students in using Python more confidently and consistently for engineering problem solving.

The findings suggest that contextually embedded, largely pre-populated notebooks with manufacturing-specific examples can meaningfully lower the barrier to Python adoption for students who have prior exposure to programming but limited confidence in applying it independently. Students engaged with the resources, reported positive perceptions of their usefulness, and expressed intent to use Python in future engineering work. At the same time, the high degree of scaffolding that made the notebooks accessible also limited opportunities for students to develop transferable coding skills, a trade-off that should be addressed in future iterations through graduated scaffolding and guided coding tasks.

The broader contribution of this work lies in demonstrating a practical, low-infrastructure approach to embedding computational tools within disciplinary laboratory courses. By building on the Jupyter Book framework and GitHub Pages, the resources remain openly accessible, version-controlled, and adaptable across future course offerings without requiring institutional hosting or

significant technical overhead. For engineering programs seeking to support programming adoption beyond standalone introductory courses, this approach offers a replicable model that prioritizes disciplinary relevance, accessibility, and iterative refinement guided by student feedback. These findings are consistent with the broader literature on computational notebooks in engineering education, which similarly identifies the tension between scaffolded accessibility and independent skill development as a central design challenge [9, 10]. Where this work contributes a practical example of how that tension can be navigated within a discipline-specific laboratory context, future work should examine whether graduated scaffolding across a sequence of notebooks produces measurable gains in student coding independence, and how similar approaches might be extended across other courses in the manufacturing engineering curriculum.

Acknowledgements

This work could not have been done without the participation of the students. We thank all those that participated by providing reflective feedback.

The study was conducted with behavioural ethics approval under certificate: REB number: H23-04100

Disclosure Statement

LLM tools such as ChatGPT and Claude were used for editing this manuscript for flow and brevity.

References

- [1] M. Hernandez-de-Menendez, C. A. Escobar Díaz, and R. Morales-Menendez, "Engineering education for smart 4.0 technology: a review," *Int. J. Interact. Des. Manuf.*, vol. 14, no. 3, pp. 789–803, 2020.
- [2] A. J. Magana, C. Vieira, F. G. Polo, J. Yan, and X. Sun, "An exploratory survey on the use of computation in undergraduate engineering education," *Proc. - Front. Educ. Conf. FIE*, pp. 94–100, 2013.
- [3] T. Koulouri, S. Lauria, and R. D. Macredie, "Teaching introductory programming: A quantitative evaluation of different approaches," *ACM Trans. Comput. Educ.*, vol. 14, no. 4, 2014.
- [4] M. A. Rubio, R. Romero-Zaliz, C. Manoso, and A. P. De Madrid, "Enhancing an introductory programming course with physical computing modules," *Proc. - Front. Educ. Conf. FIE*, vol. 2015-Febru, no. February, 2015.
- [5] E. Lahtinen, K. Ala-Mutka, and H.-M. Järvinen, "A study of the difficulties of novice programmers," *ACM SIGCSE Bull.*, vol. 37, no. 3, pp. 14–18, 2005.
- [6] M. Piteira and C. Costa, "Learning computer

- programming: Study of difficulties in learning programming,” *ACM Int. Conf. Proceeding Ser.*, pp. 75–80, 2013.
- [7] R. M. Siegfried, K. G. Herbert-Berger, K. Leune, and J. P. Siegfried, “Trends of commonly used programming languages in CS1 and CS2 learning,” *ICCSE 2021 - IEEE 16th Int. Conf. Comput. Sci. Educ.*, no. Iccse, pp. 407–412, 2021.
- [8] A. Cardoso, J. Leitão, and C. Teixeira, “Using the Jupyter Notebook as a Tool to Support the Teaching and Learning Processes in Engineering Courses,” in *The Challenges of the Digital Transformation in Education*, 2019, pp. 227–236.
- [9] J. W. Johnson, “Benefits and Pitfalls of Jupyter Notebooks in the Classroom,” *SIGITE 2020 - Proc. 21st Annu. Conf. Inf. Technol. Educ.*, no. February, pp. 32–37, 2020.
- [10] L. A. Barba *et al.*, “Teaching and Learning with Jupyter,” 2019.
- [11] M. Seddighi, D. Allanson, G. Rothwell, and K. Takroui, “Study on the use of a combination of IPython Notebook and an industry-standard package in educating a CFD course,” *Comput. Appl. Eng. Educ.*, vol. 28, no. 4, pp. 952–964, 2020.
- [12] I. Harel and S. Papert, “Situating Constructionism,” *Constructionism*, pp. 1–16, 1991.
- [13] S. A. Papert, *Mindstorms: Children, computers, and powerful ideas*. Basic books, 2020.
- [14] A. Borders, M. Earleywine, and S. J. Huey, “Predicting Problem Behaviors with Multiple Expectancies: Expanding Expectancy-Value Theory,” *Adolescence*, no. 39 (155), pp. 539–550, 2004.
- [15] I. H. Bell, J. Wronski, S. Quoilin, and V. Lemort, “Pure and Pseudo-pure Fluid Thermophysical Property Evaluation and the Open-Source Thermophysical Property Library CoolProp,” *Ind. & Eng. Chem. Res.*, vol. 53, no. 6, pp. 2498–2508, 2014.
- [16] P. Jupyter *et al.*, “Jupyter Book 2 and the MyST Document Stack,” *Proc. 24th Python Sci. Conf.*, vol. 2, pp. 173–193, 2025.
- [17] T. Kluyver *et al.*, “Jupyter Notebooks—a publishing format for reproducible computational workflows,” *Position. Power Acad. Publ. Play. Agents Agendas - Proc. 20th Int. Conf. Electron. Publ. ELPUB 2016*, pp. 87–90, 2016.
- [18] A. Aulicino and S. Bakrania, “A Python-based lab module to conduct thermodynamic cycle analysis,” in *Proceedings - Frontiers in Education Conference, FIE*, 2022, vol. 2022-October.

Appendix A: SURVEY QUESTIONS

Following is a list of survey questions that were administered using the Qualtrics platform.

The first six questions were presented with a Likert scale from ‘Strongly agree’ to ‘Strongly disagree’:

1. I am comfortable performing basic calculations in Python.
2. I am comfortable creating scatter and line plots in Python.
3. I am comfortable analyzing given data in Python.
4. I received support from the instructor/TA for using Python.
5. I will use Python for future programming projects
6. I found the Python examples shown in class/tutorials to be useful
7. When writing code, I sought help from and/or used (Select all that apply):
Instructor/TA/classmate/Generative AI (ChatGPT, Claude, etc.)/other resources (specify)
8. Your prior experience with programming (Select all that apply)
C++, Python, Matlab, C, Excel, other (Specify)
9. Your prior experience with Python.
Only in MANU 230, Through student teams, through personal projects, through high school experiences, Other (please specify)
10. Share your experiences with learning and using Python in MANU 230 (open-ended)
11. How could your learning experience be improved? (open ended)